

学号： 2104230414

沈阳建筑大学

上机报告

课程名称： 算法设计与分析

项目名称： 随机化算法

2023年秋季学期

上机时间：第 8 周 星期 二 第 3 ~ 4 节

学生姓名： 张清晨 专业班级： 计算机 2101

指导教师： 刘俊岭 成 绩：

项目名称	随机化法		学时	4
日期：2023 年 10 月 17 日		第 8 教学周		
硬件及软件				
名 称		版本、型号		
Visual C++		6.0 及以上		
上机目的与要求： 1. 问题描述，题目定义。 2. 算法实例，实例演示程序过程。 3. 程序代码，完整可运行程序，并加注释。 4. 程序验证，要求有不同的输入、输出，并截图。 5. 算法分析，时间复杂度。 本次任务： 实验 7-1：计算定积分(随机化算法) 实验 7-2：舍伍德快速排序(随机化算法) 实验 7-3：拉斯维加斯算法实现 nQueen(随机化算法)				

实验 7-1：计算定积分(随机化算法)

1. 问题描述，题目定义。

用随机投点法计算定积分。

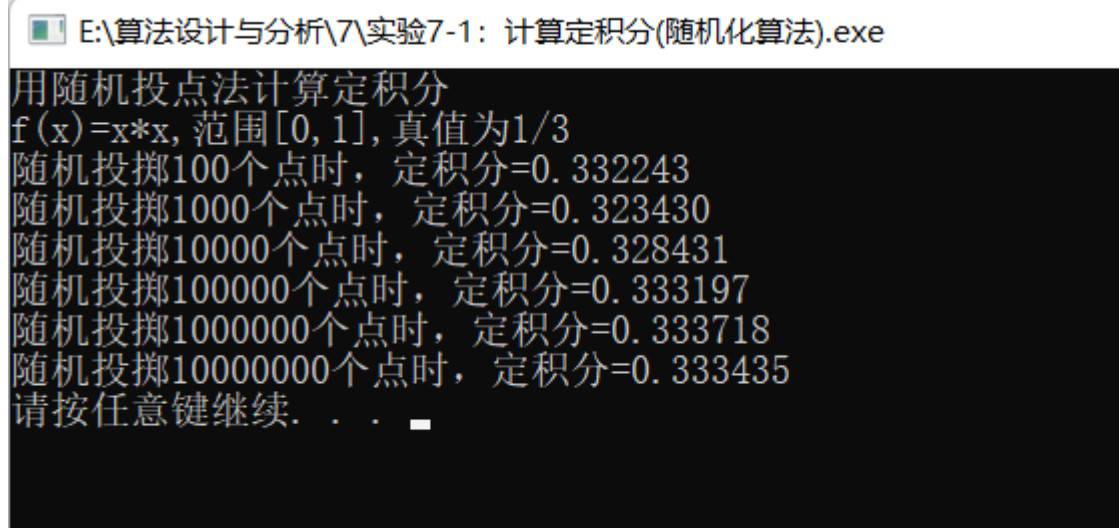
2. 算法实例，实例演示程序过程。

输入： $f(x)=x*x$ ，区间 $a = 0.0, b = 1.0$ 投点次数 $n=100$

要求：

- 1 填空完成程序；
- 2 输出不同投点次数时定积分的值。
- *3. 改变函数表达式、区间值为 $f(x)=1/3*x*x*x$ ，区间 $a = 1.0, b = 3.0$ ，计算定积分。

输出：



```
E:\算法设计与分析\八实验7-1：计算定积分(随机化算法).exe
用随机投点法计算定积分
f(x)=x*x, 范围[0, 1], 真值为1/3
随机投掷100个点时，定积分=0.332243
随机投掷1000个点时，定积分=0.323430
随机投掷10000个点时，定积分=0.328431
随机投掷100000个点时，定积分=0.333197
随机投掷1000000个点时，定积分=0.333718
随机投掷10000000个点时，定积分=0.333435
请按任意键继续. . .
```

3. 程序代码，完整可运行程序，并加注释。

```
/**
随机化算法 用随机投点法计算定积分
*/

#define _CRT_SECURE_NO_WARNINGS

#include "iostream"
#include "ctime"
#include "iomanip"

using namespace std;

const unsigned long maxshort = 65536L;
const unsigned long multiplier = 1194211693L;
const unsigned long adder = 12345L;

class RandomNumber
{
```

```

private:
    unsigned long randSeed; //当前种子
public:
    RandomNumber(unsigned long s = 0); //构造函数，默认值 0 表示由系统自动产生种子
    unsigned short Random(unsigned long n); //产生 0:n-1 之间的随机整数
    double fRandom(void); //产生[0:1)之间的随机实数
};

RandomNumber::RandomNumber(unsigned long s) //产生种子
{
    if (s == 0)
        randSeed = time(0); //用系统时间产生种子
    else
        randSeed = s; //用户提供种子
}

unsigned short RandomNumber::Random(unsigned long n) //产生 0:n-1 之间的随机整数
{
    randSeed = multiplier * randSeed + adder;
    return (unsigned short)((randSeed >> 16) % n); //高 16 位随机性较好，右移 16 位后，映射到 (0~n-1) 范围内
}

double RandomNumber::fRandom(void) //产生[0,1)之间的随机整数
{
    return Random(maxshort) / double(maxshort); //产生 0~(maxshort-1)间的随机整数，在除以 maxshort
}

double Darts(int n, double a, double b);
double f(double x);

int main()
{
    double a = 0.0, b = 1.0;
    double integral;
    long num = 100;
    printf("用随机投点法计算定积分\n");
    printf("f(x)=x*x,范围[0,1],真值为 1/3\n");

    for (int i = 1; i < 7; i++){
        integral = Darts(num, a, b);
        printf("随机投掷%d 个点时， ", num);
        printf("定积分=%f\n", integral);
        num = num * 10;
    }
}

```

```

    }

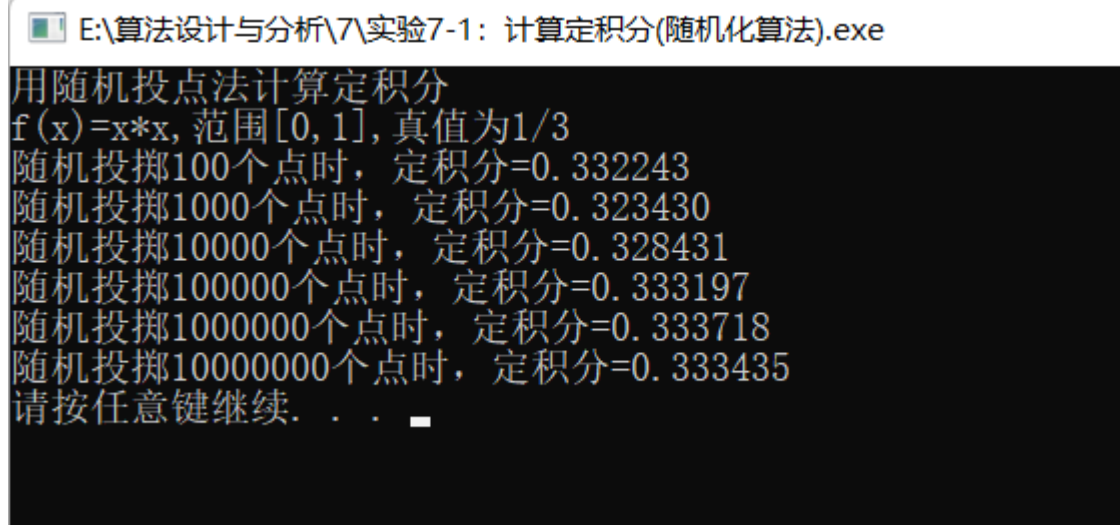
    system("PAUSE");
    return 0;
}
/*
* 基本思想是在矩形区域内随机均匀投点，求出由这些点
* 产生的函数值的算术平均值，再乘以区间宽度，即可得
* 出定积分的近似解
*/
double Darts(int n, double a, double b)
{
    static RandomNumber dart;
    double sum = 0.0;
    for (int i = 0; i < n; i++)
    {
        double x = (b - a) * dart.fRandom() + a; //产生[a,b]之间的随机数
        sum = sum + f(x);
    }
    return (b - a) * sum / n;
}

double f(double x)
{
    return x * x;
}

```

4. 程序验证，要求有不同的输入、输出，并截图。

示例 1：输入的例子为： $f(x)=1/3 * x * x$ ，区间 $a = 1.0, b = 4.0$ 投点次数 $n=100、1000、10000、100000、1000000$ 时，进行结果输出。



```

E:\算法设计与分析\八实验7-1：计算定积分(随机化算法).exe
用随机投点法计算定积分
f(x)=x*x, 范围[0, 1], 真值为1/3
随机投掷100个点时，定积分=0.332243
随机投掷1000个点时，定积分=0.323430
随机投掷10000个点时，定积分=0.328431
随机投掷100000个点时，定积分=0.333197
随机投掷1000000个点时，定积分=0.333718
随机投掷10000000个点时，定积分=0.333435
请按任意键继续. . .

```

示例 2： $f(x)=1/3*x*x*x$ ，区间 $a = 1.0, b = 3.0$

```
用随机投点法计算定积分
f(x)=1/3*x*x*x, 范围[1, 3], 真值为20/3
随机投掷100个点时, 定积分=7.474533
随机投掷1000个点时, 定积分=6.585117
随机投掷10000个点时, 定积分=6.620954
随机投掷100000个点时, 定积分=6.690776
随机投掷1000000个点时, 定积分=6.670679
随机投掷10000000个点时, 定积分=6.667175
请按任意键继续. . .
```

5. 算法分析, 时间复杂度。

该题使用了数值随机算法。利用随机数发生器的算法, 算法执行的某些选择依赖于随机数发生器所产生的随机数.其时间复杂度为 $O(n)$, n 为投点尝试数。使用了长度为 1 的记录变量, 所以空间复杂度为 1。

- 时间复杂度: $O(n)$
- 空间复杂度: $O(1)$

实验 7-2: 舍伍德快速排序(随机化算法)

1. 问题描述, 题目定义。

利用舍伍德算法实现快速排序。

2. 算法实例, 实例演示程序过程。

示例:

输入 100 数值。

要求:

- 1 填空完成程序;
- 2 输出排序结果。
- *3 设计一个划分算法, 利用 3 个随机数的中位数作为 pivot。

输出:

要求 1, 2

```
图1 E\算法设计与分析\实验7-2: 舍伍德快速排序(随机化算法).exe
原序列:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66
67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
第一次洗牌:
26 4 19 53 75 47 48 55 97 38 35 23 100 62 67 1 73 29 60 6 71 36 53 9 43 68 59 46 32 72 20 82 12 52 33 21 28 8 70 5 74 86 2 10 24 88 69 83 63 39 95 89 27 31 17 78 16 90 25 22 13 7 93 98 87 8
1 56 76 34 64 42 96 44 50 91 18 80 92 84 66 51 61 15 49 99 3 85 40 54 94 45 77 37 11 41 65 30 14 57 79
第二次洗牌:
68 58 60 90 91 69 83 17 30 8 33 53 79 7 56 26 44 32 22 47 42 21 27 97 2 76 25 88 82 96 6 61 23 89 12 71 46 55 64 75 50 3 4 38 9 40 34 15 93 70 41 54 59 20 73 92 1 94 43 36 100 48 37 14 85 5
1 78 13 52 98 85 65 10 39 45 29 66 77 49 81 95 13 67 63 57 19 99 5 31 11 24 80 28 35 74 87 72 62 16 84
选择的pivot位置, 值序列为:
78:49, 10:33, 10:9, 2:1, 6:7, 4:3, 4:6, 3:4, 16:11, 30:16, 13:12, 13:15, 12:13, 26:25, 18:22, 19:17, 19:21, 18:18, 18:20, 22:24, 30:27, 30:30, 27:29, 30:31, 46:34, 46:37, 34:36, 43:38, 46:3
9, 40:42, 39:41, 46:43, 46:44, 46:46, 46:47, 92:90, 67:87, 74:76, 49:60, 57:56, 53:50, 53:52, 53:54, 57:59, 56:57, 73:73, 70:70, 61:65, 62:63, 60:61, 67:69, 66:68, 65:67, 70:72, 73:74, 84:8
4, 81:78, 81:83, 80:79, 80:80, 80:81, 84:85, 87:88, 98:92, 94:98, 95:96, 93:93, 93:94, 98:99,
排序后:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66
67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
请按任意键继续. . .
```

要求 3:

```
图1 E\算法设计与分析\实验7-2: 舍伍德快速排序(随机化算法).exe
72:72,
k1 = 74 k2 = 74 k3 = 75
k = 74
74:77,
k1 = 75 k2 = 75 k3 = 74
k = 75
75:75,
k1 = 79 k2 = 79 k3 = 85
k = 79
79:81,
k1 = 84 k2 = 85 k3 = 83
k = 84
84:85,
k1 = 81 k2 = 81 k3 = 82
k = 81
81:82,
k1 = 83 k2 = 83 k3 = 82
k = 83
83:83,
k1 = 88 k2 = 97 k3 = 94
k = 94
94:94,
k1 = 90 k2 = 90 k3 = 91
k = 90
90:88,
k1 = 91 k2 = 92 k3 = 90
k = 91
91:92,
k1 = 88 k2 = 88 k3 = 89
k = 88
88:89,
k1 = 90 k2 = 90 k3 = 89
k = 90
90:91,
k1 = 97 k2 = 97 k3 = 98
k = 97
97:99,
k1 = 95 k2 = 97 k3 = 96
k = 96
96:96,
k1 = 97 k2 = 97 k3 = 96
k = 97
97:97,
排序后:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66
67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
请按任意键继续. . .
```

3. 程序代码，完整可运行程序，并加注释。

/*exp7-4 QuickSortSherwood

舍伍德快速排序。虽然快速排序平均复杂度可以看做 $n\log n$.

但算法复杂度与输入实例之间有关系，未必都是平均时间复杂度。

这是采用舍伍德快速排序的原因，消除掉算法复杂度与输入实例之间的关系。

*/

#define _CRT_SECURE_NO_WARNINGS

#include<iostream>

#include<stdlib.h>

#include<time.h>

#include<cstdio>

#include<algorithm>

#define MAX 100

using namespace std;

int Partition(int a[], int low, int high)

{

int key = a[low];

int i = low, j = high + 1;

while (1)

{ while (a[++i]<key&&i <= high);

```

        while (a[--j]>key&& j >= low);
        if (i >= j) break;
        if (a[i] != a[j])
            swap(a[i], a[j]);
    }
    if ((j != low) && (a[low] != a[j]))
        swap(a[low], a[j]);
    return j;
}

//随机得到一个位置进行划分操作
int RandPartition(int a[], int low, int high)
{
    srand(time(NULL));
    //随机生成一个 low,high 之间的位置 k
    int k = rand() % (high - low + 1) + low;
    cout << k << ":" << a[k] << ", ";
    //将 a[k]作为 pivot
    swap(a[k], a[low]);
    k = Partition(a, low, high);
    return k;
}

void Quick_sort(int a[], int low, int high)
{
    if (low < high)
    {
        int k = RandPartition(a, low, high);
        Quick_sort(a, low, k - 1);
        Quick_sort(a, k + 1, high);
    }
}

// 随机洗牌算法
void random_shuffle(int a[]) {

    srand(time(NULL));
    for (int i = MAX - 1; i > 0; i--) {
        int j = rand() % (i + 1);
        int temp = a[i];
        a[i] = a[j];
        a[j] = temp;
    }
}

int main()
{

```



```

int i;
int a[MAX];
for (i = 0; i < MAX; i++)
    a[i] = i + 1;
cout << "原次序: ";
cout << endl;
for (i = 0; i < MAX; i++)
    cout << a[i] << " ";
cout << endl;
random_shuffle(a); //To randomize the array
//cout << "排序前: ";
cout << "第一次洗牌: ";
cout << endl;
for (i = 0; i < MAX; i++)
    cout << a[i] << " ";
cout << endl;
random_shuffle(a); //To randomize the array
//cout << "排序前: ";
cout << "第二次洗牌: ";
cout << endl;
for (i = 0; i < MAX; i++)
    cout << a[i] << " ";
cout << endl;

cout << "选择的 pivot 位置, 值序列为: ";
cout << endl;

Quick_sort(a, 0, MAX-1);
cout << endl;

cout << "排序后: ";
cout << endl;
for (i = 0; i < MAX; i++)
    cout << a[i] << " ";
cout << endl;

system("PAUSE");
return 0;
}

```

4. 程序验证, 要求有不同的输入、输出, 并截图。

输入的数据为 100 个乱序的数字, 对其进行快速排序。看是否能按照数字的大小进行排序, 第一种输出为产生一个随机数, 即为 pivot, 以此进行快速排序, 运行程序得到以下截图:

```
E:\算法设计与分析\实验7-2: 舍伍德快速排序(随机化算法).exe
72:72,
k1 = 74 k2 = 74 k3 = 75
k = 74
74:77,
k1 = 75 k2 = 75 k3 = 74
k = 75
75:75,
k1 = 79 k2 = 79 k3 = 85
k = 79
79:81,
k1 = 84 k2 = 85 k3 = 83
k = 84
84:85,
k1 = 81 k2 = 81 k3 = 82
k = 81
81:82,
k1 = 83 k2 = 83 k3 = 82
k = 83
83:83,
k1 = 88 k2 = 97 k3 = 94
k = 94
94:94,
k1 = 90 k2 = 90 k3 = 91
k = 90
90:88,
k1 = 91 k2 = 92 k3 = 90
k = 91
91:92,
k1 = 88 k2 = 88 k3 = 89
k = 88
88:89,
k1 = 90 k2 = 90 k3 = 89
k = 90
90:91,
k1 = 97 k2 = 97 k3 = 98
k = 97
97:99,
k1 = 95 k2 = 97 k3 = 96
k = 96
96:96,
k1 = 97 k2 = 97 k3 = 96
k = 97
97:97,
排序后:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66
67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
请按任意键继续. . .
```

5. 算法分析，时间复杂度。

本题是关于快速排序的问题，在传统的快速排序算法的基础上，使用舍伍德(Sherwood)算法进行优化，而舍伍德算法随机地选择一个数组元素作为划分基准，引入随机性将它改造成一个舍伍德算法，用来消除或减少问题的不同实例之间在计算时间上的差别。舍伍德算法的精髓不是避免算法的最坏情况行为，而是设法消除这种最坏行为与特定实例之间的关联性。其时间复杂度为 $O(n\log n)$ 。空间复杂度，本次题只用了常量级的变量进行存储，所以空间复杂度为 $O(1)$ 。所以

- 时间复杂度： $O(n\log n)$
- 空间复杂度： $O(1)$

实验 7-3：拉斯维加斯算法实现 nQueen(随机化算法)

1. 问题描述，题目定义。

nQueen 随机放置位置策略和回溯法相结合求解 n 皇后问题。

拉斯维加斯算法不会得到不正确的解。一旦用拉斯维加斯算法找到一个解，这个解就一定是正确解。但有时用拉斯维加斯算法找不到解。与蒙

特卡罗算法类似，拉斯维加斯算法找到正确解的概率随着它所用的计算时间的增加而提高。对于所求解问题的任一实例，用同一拉斯维加斯算法反复对该实例求解足够多次，可使求解失败的概率任意小。

N 皇后的求解方法：

消极：LV 算法过于消极，一旦失败，从头再来。

乐观：回溯算法过于乐观，一旦放置某个皇后失败，就进行系统回退一步的策略，而这一步往往不一定有效

折中：一般情况下更好，先用 LV 方法随机的放置前若干个结点，然后使用回溯法放置后若干个结点

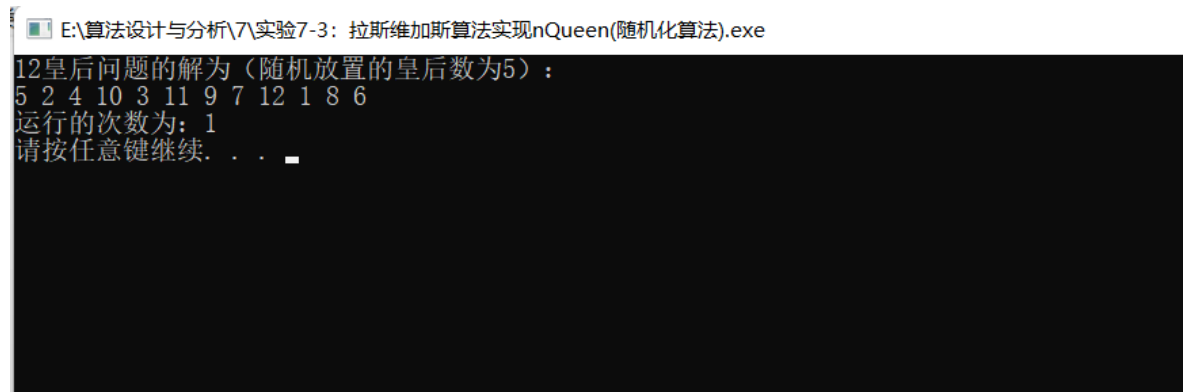
2. 算法实例，实例演示程序过程。

示例：皇后数 n=12 随机放置皇后数 stop=5

要求：

- 1 填空完成程序；
- 2 输出放置结果，运行次数。
- *3 改变皇后数与随机放置皇后数，输出结果。

输出：



```
E:\算法设计与分析\7实验7-3: 拉斯维加斯算法实现nQueen(随机化算法).exe
12皇后问题的解为（随机放置的皇后数为5）：
5 2 4 10 3 11 9 7 12 1 8 6
运行的次数为: 1
请按任意键继续. . .
```

3. 程序代码，完整可运行程序，并加注释。

/*nQueen 随机放置位置策略和回溯法相结合求解 n 皇后问题.

*/

```
#define _CRT_SECURE_NO_WARNINGS
```

```
#include <iostream>
```

```
#include <vector>
```

```
#include <string>
```

```
#include<math.h>
```

```
#include<time.h>
```

```

using namespace std;
//利用线性同余法产生的伪随机数
const unsigned long maxshort = 65536;
const unsigned long multipliter = 1194211693L;
const unsigned adder = 12345L;
class RandomNumber{
private:
    unsigned long randSeed;
public:
    RandomNumber(unsigned long s = 0);
    unsigned short Random(unsigned long n);
    double fRandom(void);
};
RandomNumber::RandomNumber(unsigned long s){
    if (s == 0)
        randSeed = time(0);
    else
        randSeed = s;
}
unsigned short RandomNumber::Random(unsigned long n){
    randSeed = multipliter*randSeed + adder;
    return (unsigned short)((randSeed >> 16) % n);
}
double RandomNumber::fRandom(void){
    return Random(maxshort) / double(maxshort);
}

class Queen{
    friend bool nQueen(int);
private:
    bool Place(int k);
    bool Backtrack(int t);
    bool QueensLV(int stop);
    int n, *x, *y;
};
bool Queen::Place(int k){    //测试皇后 k 置于第 x[k]列的合法性
    for (int j = 1; j<k; j++)
        if ((abs(k - j) == abs(x[j] - x[k])) || (x[j] == x[k]))
            return false;
    return true;
}
bool Queen::Backtrack(int t){
    if (t>n){
        for (int i = 1; i <= n; i++)
            y[i] = x[i];
    }
}

```

```

        return true;
    }
    else{
        for (int i = 1; i <= n; i++){
            x[t] = i;
            if (Place(t) && Backtrack(t + 1))
                return true;
        }
    }
    return false;
}

bool Queen::QueensLV(int stop){//随机放置 n 个皇后的拉斯维斯加算法
    RandomNumber rnd;    // 随机数产生器
    int k = 1;            //下一个放置的皇后的编号
    int count = 1;
    while ((k <=stop) && (count>0)){
        count = 0;
        for (int i = 1; i <= n; i++){//遍历第 k 行所有可以放置的列号，用 y 保存，并用
count 记录有多少个位置可以放置
            x[k] = i;
            if (Place(k))
                y[count++] = i;
        }
        if (count>0)    //利用上面保存的可以放置的列，然后随机取其中一列来放置
第 k 行的皇后。是 Las Vegas 思想
            x[k++] = y[rnd.Random(count)];//随机位置
    }
    return (count>0); //count>0 表示放置成功
}

bool nQueen(int n){
    Queen X;
    X.n = n;
    int *p = new int[n + 1];
    int *q = new int[n + 1];
    for (int i = 0; i <= n; i++){
        p[i] = 0;
        q[i] = 0;
    }
    X.y = p;
    X.x = q;
    // 设置随机放置皇后的个数
    int stop = 5;
    if (n>15)
        stop = n - 15;
    bool found = false;
    while (!X.QueensLV(stop));
}

```

```

// 算法的回溯搜索部分
if (X.Backtrack(stop + 1)){
    for (int i = 1; i <= n; i++)
        cout << p[i] << " ";
    found = true;
}
cout << endl;
delete[] p;
delete[] q;
return found;
}

int main(){
    int n = 12;
    int k=1;

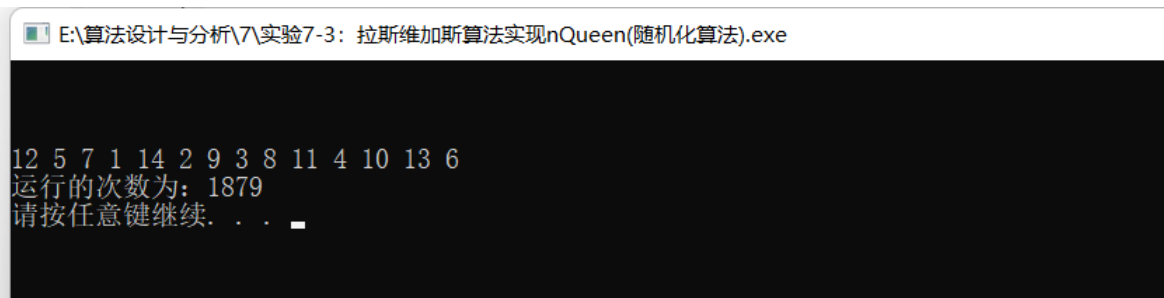
    cout << n << "皇后问题的解为（随机放置的皇后数为 5）：" << endl;
    while (!nQueen(n)){
        k++;
    }

    cout << "运行的次数为：" << k << endl;
    system("PAUSE");
    return 0;
}

```

4. 程序验证，要求有不同的输入、输出，并截图。

更改皇后数，求解 14 皇后问题，随机放置的皇后数为 7. 结果如图所示：



5. 算法分析，时间复杂度。

时间复杂度： $O(n^2)$

空间复杂度： $O(n)$

结果分析和总结：

随机化算法的基本特征就是对所求解问题的同一实例用同一随机化算法求解两次可能得到完全不同的效果。使用随机策略有时候可以对原本的算法进行一定程度上的优化，从而改善算法。

本节课的学习中随机化算法一共有四种，分别是概率数值法、舍伍德算法、拉斯维加斯算法、蒙特卡洛法，对于四种随机化算法的比较，它们各有特点，例如有时候拉斯维加斯算法找不到解，而另外三种都能找到解，而随机数值法只能求得近似解，而蒙特卡洛算法一定能找到解，但这个解不一定是正确解，舍伍德算法能找到解、而且保证这个解是正确解，但是舍伍德算法不一定能优化原来的算法，它只是用来消除输入数据与最坏情况的关联性。所以四种算法各有特点，需要结合实际的去使用对应的算法来优化。

本 次 上 机 成 绩	项目	操作	报告书写	出勤和课堂纪律	其他
	得分				
	成绩合计：				

评语：

教师签字：

日期：